

ReviewPilot - AI PR Review & Test Planner

Prepared for: DriveNets AI Specialist Engineer Process

Candidate: Ilay Romi

Date: May 2026

Repository: <https://github.com/IlayRomi/reviewpilot-ai-pr-review>

Local demo commands: Provided below.

1. Executive Summary

ReviewPilot is a command-line tool I built with Claude Code to support pull-request review. Given a local unified `.diff` file, it produces a structured Markdown report that helps a reviewer understand what changed, where the risk is, and what should be tested.

My goal was not to build a naive LLM wrapper. I intentionally separated deterministic engineering logic from AI-assisted suggestions:

- **Deterministic layer:** parse the diff, classify files, compute transparent risk signals, and build a structured report context.
- **AI-assisted layer:** generate regression hypotheses, test suggestions, and a reviewer checklist from that structured context.

The most important design decision is that the AI client never receives raw diff text. It receives only an `AnalysisContext`: file summaries, roles, risk scores, and detected signal labels. This makes the core logic testable, the AI input inspectable, and the output easier to evaluate.

Outcome: the MVP is working end-to-end. It includes a CLI (`python -m reviewpilot`), four realistic sample diffs, a generated sample report, full README and development log, and **451 passing tests**. The current AI layer uses `MockAIClient` intentionally: it is deterministic, offline, and satisfies the same `AIClient` protocol that a real Anthropic client could implement later.

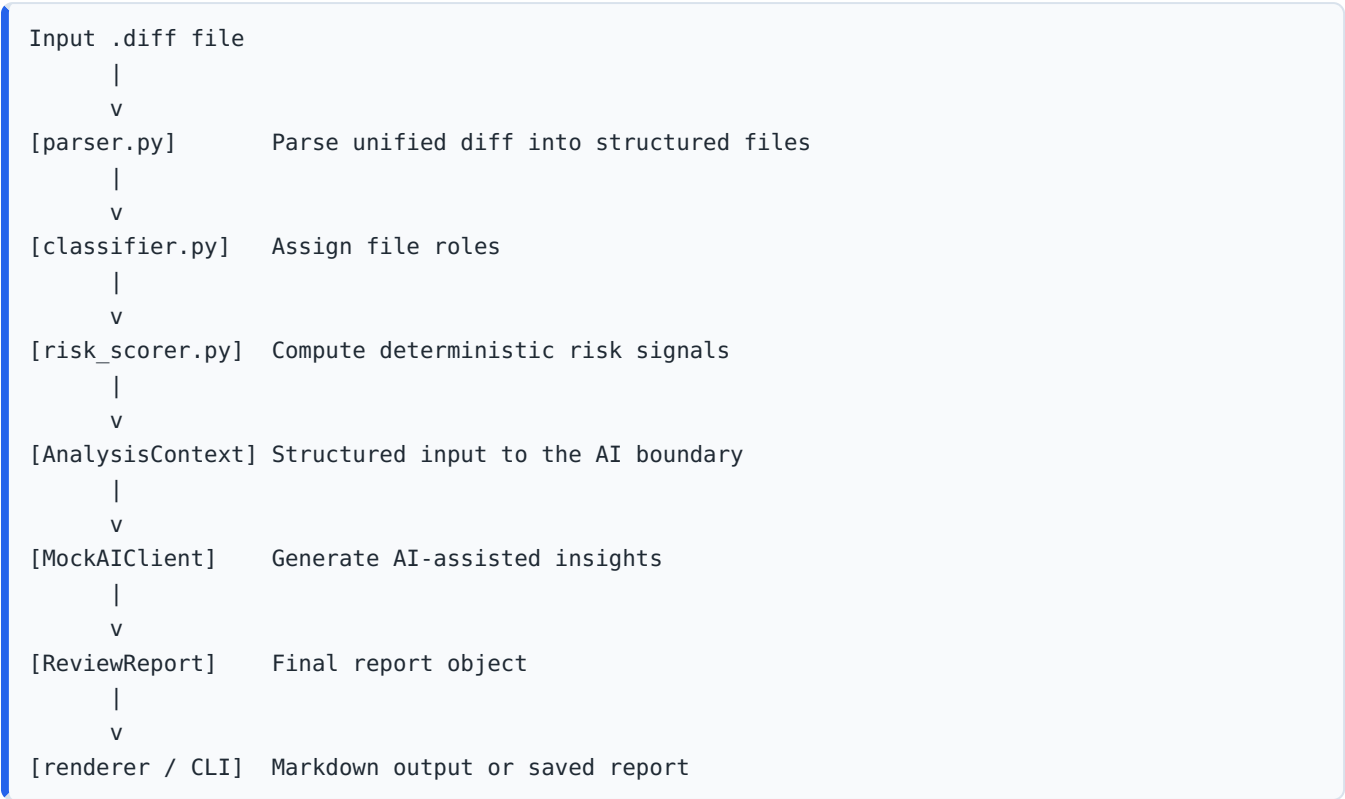
2. Problem and Approach

Code reviewers often receive a multi-file diff without a clear signal about where to focus. They need to answer four questions quickly: what changed, which files are risky, what tests are missing, and what should a human reviewer verify manually.

Sending a raw diff directly to an LLM is not sufficient. It is difficult to test, can produce non-deterministic results, and may hallucinate behavior not present in the diff. ReviewPilot addresses this by first producing a deterministic analysis, then using AI only for clearly labeled suggestions.

The generated report separates computed facts from hypotheses. Risk levels come from explicit rules, while AI-assisted sections are framed as suggestions for human validation.

3. Architecture



Component	Responsibility	Verification
models.py	Typed dataclasses and enums used across the pipeline	Model construction and property tests
parser.py	Parses unified diff text into structured diff objects	Unit tests for change types and invalid diffs
classifier.py	Maps paths to roles such as SOURCE, TEST, MIGRATION, DOCS	Unit tests for patterns, edge cases, and precedence
risk_scorer.py	Computes deterministic risk signals and risk levels	Unit tests for every rule and boundary
ai_client.py	Defines AIClient protocol and deterministic MockAIClient	Grounding, determinism, and protocol tests
report_builder.py	Orchestrates parse -> classify -> score -> context -> AI insights	Integration tests
renderer.py	Converts ReviewReport to Markdown	Renderer tests for sections, tables, and escaping
cli.py	User-facing command-line interface	CLI tests for stdout, file output, and errors

Deterministic vs AI-assisted design

Layer	Produces	Why it matters
Deterministic	File roles, risk signals, scores, report structure	Fully testable and explainable
AI-assisted	Regression hypotheses, test suggestions, reviewer checklist	Useful for reviewer thinking, but labeled as suggestions

The `AnalysisContext` boundary gives four benefits: reproducible prompts, inspectable AI input, reduced exposure of raw code, and a clean path to replacing `MockAIClient` with a real Anthropic client later.

4. Claude Code Development Process

I used Claude Code as an engineering assistant, not as an unchecked code generator. The workflow was: plan first, implement one module per commit, require tests with each module, verify before committing, and use git checkpoints.

Stage	Result
Scaffold	Project structure, <code>pyproject.toml</code> , smoke test
Models	Dataclasses and enums
Parser	Unified diff parser
Classifier	Path-based file role classifier
Risk scorer	Deterministic weighted risk rules
AI client	<code>AIClient</code> protocol and <code>MockAIClient</code>
Report builder	Full pipeline orchestration
Renderer	Markdown report generation
CLI	User-facing command line tool
Documentation	README, dev log, sample diffs, sample report, and final project report

Prompting patterns used

I did not ask Claude to build everything at once. I used constrained prompts:

- **Planning prompt:** define architecture, data models, risks, and implementation order without coding.
- **Module prompt:** implement one file only, with explicit constraints and no unrelated changes.
- **Testing prompt:** create focused tests for specific behavior and edge cases.
- **Failure-analysis prompt:** explain the root cause before changing code.
- **Documentation prompt:** write for a technical reviewer, with accurate limitations and no exaggerated claims.

A representative example of a module prompt:

```
Proceed with Commit #5 only: implement deterministic risk scoring in risk_scorer.py.  
Do not modify parser, classifier, ai_client, report_builder, renderer, or cli.  
Add tests covering each signal and boundary condition, then run pytest.
```

This made the AI output easier to review and kept the work focused on a testable engineering artifact.

5. Testing Strategy

I designed the test suite around four layers that build confidence from individual module behavior to the final user-facing workflow. Unit tests per module verify each deterministic component in isolation: parsing edge cases and change types, classification precedence rules, every risk scoring rule and score-band boundary, and `MockAIClient` grounding and determinism. Integration tests for `report_builder` and `renderer` verify that the modules compose correctly end-to-end — confirming that parse, classify, score, and render work together as a pipeline using inline diff fixtures. CLI tests verify the user-facing workflow: stdout output, file writing, error exit codes, and edge cases like missing files or empty diffs. Manual checks on the four sample diffs confirmed that the assembled report is readable, that the correct signals fire for realistic inputs, and that the AI-assisted sections are clearly labeled. Together, these layers build confidence from individual module behavior up to the final user-facing artifact.

All AI-related tests use `MockAIClient`. This avoids network calls, API keys, non-determinism, and token cost while still testing the AI boundary and report flow.

Milestone	Tests passing
Parser	89
Classifier	178
Risk scorer	244
AI client	290
Report builder	350
Renderer	415
Final CLI/project	451

I did not implement golden-output evaluation for a real LLM because the current MVP does not call a live model. A golden evaluation set is listed as future work for the real API version.

6. Example of AI Output I Evaluated and Corrected

The clearest issue caught during development was in the parser tests. Claude generated a test fixture where one test expected three context lines in a unified diff, but the parser returned two. The failing test surfaced this before anything was committed, giving me the opportunity to investigate the cause rather than just fix the symptom. Instead of changing the parser immediately, I investigated whether the failure was caused by the implementation or by the AI-generated fixture.

The root cause was the fixture: in unified diff format, a blank context line must be represented as a line starting with one space (`" "`), not as a bare empty line (`""`). The parser was correct; the test fixture was malformed.

Resolution: I fixed the fixture and kept the parser behavior unchanged. Weakening the parser would have made the test pass while making the implementation less correct.

Other verification issues were handled the same way: I created a local virtual environment when `pytest` was missing globally, used `pytest` instead of long shell spot-checks, and caught an initially untracked `__main__.py` file with `git status` before finalizing the CLI commit.

7. Critical Reflection

Claude was very effective for structure, boilerplate, tests, and documentation once the task was well constrained. It accelerated implementation, but it did not replace engineering judgment.

Claude was strong at	I retained ownership of
Generating consistent module structure and docstrings	Defining and preserving the deterministic/AI boundary
Creating broad initial test coverage from explicit requirements	Limiting scope to a realistic MVP
Helping diagnose failures when asked to reason first	Deciding not to add a real API client yet
Drafting documentation for a stated audience and tone	Reviewing tests for meaningful assertions and commit hygiene

The main lesson is that Claude Code is strongest when directed with clear constraints. The developer still owns the architecture, verification strategy, and final quality bar.

8. What I Chose Not To Test

Not tested	Why acceptable for MVP
Real LLM output quality	Current version uses <code>MockAIClient</code> ; real model evaluation needs a golden-output suite
Very large diffs	MVP targets typical PR diffs, not multi-megabyte changes
Full malformed diff recovery	Parser supports standard <code>git diff</code> output and clear error cases
Runtime behavior of changed code	<code>ReviewPilot</code> analyzes diffs; it does not execute the target application
Semantic code understanding	Risk scoring is heuristic and keyword-based by design
GitHub/GitLab integration	Local <code>.diff</code> files were the intended scope

The accepted risk is that the tool provides review guidance, not proof of correctness. It helps reviewers focus, but it does not replace tests, code ownership, or manual review.

9. Limitations and Future Work

Current limitations:

- Local `.diff` files only.
- `MockAIClient` only; no live LLM API in the current MVP.
- Heuristic risk scoring; no AST or control-flow analysis.
- No repository history, dependency graph, or coverage data.
- Markdown output only.

Future improvements:

- Add a real `AnthropicAIClient` behind the existing `AIClient` protocol.
- Add GitHub PR integration and PR comments.
- Make risk rules configurable through `reviewpilot.toml`.
- Enrich `AnalysisContext` with repository history and test coverage.
- Add a golden-output evaluation suite for real LLM responses.
- Add CI integration and richer output formats such as HTML or SARIF.

10. Demo

Run the project locally:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -e ".[dev]"
```

Generate a sample report:

```
python -m reviewpilot examples/auth_change.diff --output reports/sample_report.md
```

Run all tests:

```
python -m pytest tests/ -v
```

Expected result: **451 passed**.

Sample inputs are under `examples/`; the generated report is committed at `reports/sample_report.md`.

11. Conclusion

ReviewPilot reflects my approach to AI-assisted engineering work: start from a clear problem, define strict boundaries, use Claude Code to accelerate implementation, and verify every output before accepting it.

The scope is intentionally modest, but the MVP is complete: it has a working CLI, realistic examples, deterministic risk analysis, AI-assisted suggestions through a clean protocol boundary, documentation, and 451 passing tests.

Claude was useful as a collaborator for constrained tasks. The architecture, scope, testing strategy, and final quality decisions remained mine.

ReviewPilot - AI-assisted engineering project, May 2026.